

Анализ зависимостей

Объём кода, использующийся на вашей странице, может увеличить как время до первой отрисовки, так и время до интерактивности. Обычно, если кода на странице много, то большая его часть не используется. В этой статье мы научимся находить неиспользуемый код и того, кто его загрузил.

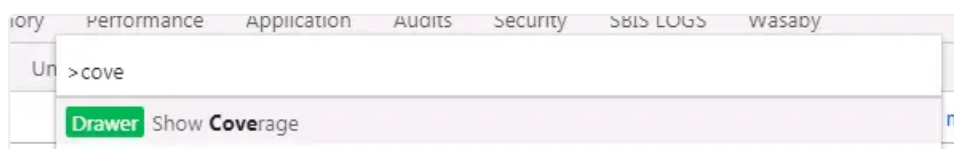
Для начала научимся находить неиспользуемые модули, т.е. те модули, которые были загружены на страницу, но не были запрошены другими модулями. Например, вы можете подключить один модуль и вместе с ним получить в пакете ещё десять.

Поиск неиспользуемых модулей

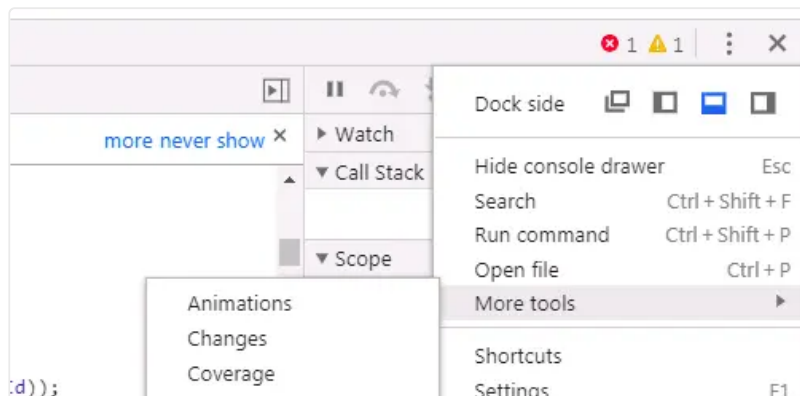
Подробно о поиске неиспользуемых модулей и анализе зависимостей описано [здесь](#).

Поиск неиспользуемого кода внутри модулей

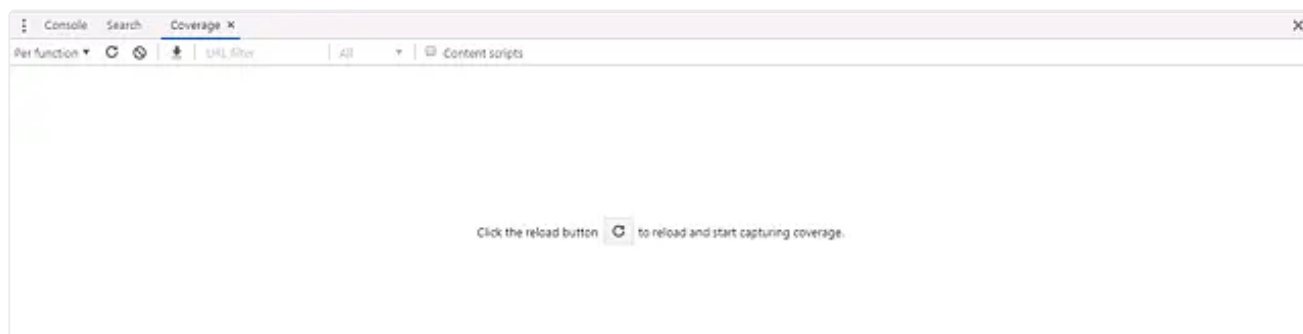
Лишний трафик бывает не только из-за неиспользуемых модулей. Бывает что модуль используется, но используется не весь. Находить неиспользуемые участки кода поможет вкладка Coverage в Chrome Dev Tools. Найти её можно через меню команд (Ctrl+Shift+P/Cmd+Shift+P):



Или в дополнительных инструментах:



Открыв вкладку, вы увидите такую картину:



После нажатия на кнопку страница перезагрузится и вы получите список файлов и сколько кода использовано в каждом из них:

URL	Type	Total Bytes	Unused Bytes	Usage Visualization
https://online.sbis.ru/.../online-superbundle.package.min.js?x_module=20.3100-1318	JS (per func...	855 078	289 174 33.8 %	
https://online.sbis.ru/resources/Controls/ctrl.min.js?x_module=20.3100-1318	JS (per func...	464 223	187 497 40.4 %	
https://online.sbis.ru/res.../core-compatible.package.min.js?x_module=20.3100-1318	JS (per func...	138 370	129 188 93.4 %	
https://online.sbis.ru/resources/Not.../noticeControls.min.js?x_module=20.3100-1318	JS (per func...	154 846	117 097 75.6 %	
https://online.sbis.ru/resou.../vdom-sidebar.package.min.js?x_module=20.3100-1318	JS (per func...	199 042	116 995 58.8 %	
https://online.sbis.ru/resources/Controls/input.min.js?x_module=20.3100-1318	JS (per func...	173 166	94 185 54.5 %	
https://online.sbis.ru/resources/NoticeCen.../panelView.min.js?x_module=20.3102-21	JS (per func...	143 052	81 730 56.3 %	

При двойном нажатии на файл, он откроется во вкладке Sources и можно будет посмотреть какие конкретно строки исполнялись:

```

1  define("wml!LongOperations/Starter/Controller", ["View/Executor/TClosure", "
2      function t() {
3          debugger
4      }
5      var r = Array.prototype.slice.call(arguments)
6          , y = {}
7          , v = {};
8      y["Controls/popup"] = r[2];
9      var n = function e(t, r, n, o, i) {
10         var a = 0
11             , l = 0
12             , p = f.validateNodeKey(r && r.key)
13             , s = {
14                 id: [],
15                 def: void 0
16             }
17         , c = f.configResolver.calcParent(this, "undefined" === typeof cur
18         , d = f.getMarkupGenerator(o);
19         try {
20             var u = d.joinElements([d.createTag("div" f

```

По этим данным можно найти модули, от использования которых можно избавиться. Далее нужно через расширение Wasaby Developer Tools находить кто их запросил и действовать по уже описанному алгоритму.